

Hello, I'm

김수진

풀사이클 개발자

요구사항 분석부터 DB 프로시저까지 한 흐름으로 설계하고 운영합니다.

공공 SI/SM에서 시작해 사내 업무 시스템 재구축과 결제 연동을 거쳐, 지금은 자사 B2C 서비스와 고객사 플랫폼의 개발·운영·배포를 함께 맡고 있습니다.

EXPERIENCE

3년 8개월 · 회사 3곳
2022.03 — 현재

STACK

React · TypeScript · Next.js
C#.NET · Java · MSSQL · AWS

FOCUS

운영 흐름 · 데이터 구조
결제 연동 · 배포 자동화

CONTACT

su_042@daum.net
github.com/Nuuuing

소개 *about me*

사용자 경험과 운영 효율을 함께 개선하는 개발자

● 일하는 방식

공공 SI/SM, 플랫폼 서비스, 사내 업무 시스템, 자사 B2C 서비스 등 서로 다른 환경과 도메인을 경험하며, 새로운 비즈니스를 마주해도 빠르게 구조를 분석하고 설계하는 적응력을 길러왔습니다. 프로젝트 환경에 맞춰 **Front · Back · DB · 인프라** 영역에 한계를 두지 않고 다루며, 단순 기능 개발을 넘어 아키텍처 설계와 인프라 구축, 배포 프로세스까지 필요에 따라 유연하게 챙겨왔습니다. 이러한 유연함과 치밀함을 바탕으로 비즈니스의 성장과 운영 효율을 함께 개선해 나가는 데 몰입합니다.

3년 8개월

경력 · 회사 3곳
2022.03 — 현재

99→70%

피크 DB CPU
실행계획 · 인덱스 정비

15

담당 업무 · 프로젝트
회사 11 · 개인·팀 5

● 학력 · 교육

2023.08 — 2027.02

학사 · 졸업예정

한국방송통신대학교

컴퓨터과학 — 자료구조,
알고리즘, 운영체제, AI 등 CS

2024.05 — 2025.01

수료

디벨로켓

메타버스 플랫폼 게임
개발자과정 — C#, Unity,
AR/XR

2016.03 — 2018.02

전문학사

수원여자대학교

모바일미디어(SW) — 웹 기초,
DB, Java, Android, CS

기술 스택 *stack*

화면 구현부터 서버 · DB · 배포까지 실무와 사이드 프로젝트에서 직접 사용한 기술입니다

FRONTEND

React TypeScript Next.js ReactQuery
Zustand jQuery Tailwind CSS Motion

상태 관리와 반응형 UI, SSG · 동적 라우팅 — 레거시
유지보수부터 모던 인터랙션까지

BACKEND

C# ASP.NET MVC Java Spring Boot
NestJS JSP PHP OpenAPI

업무 시스템 서버 · 결제 PG API · ERP 연동, 공공 데이터
OpenAPI 설계 · 외부 API 연동

DATABASE

MSSQL MySQL Oracle MariaDB
PostgreSQL Firestore

프로시저 작성 · 개선, 실행 계획 분석, 대용량 거래 데이터 처리,
인덱스 정비

INFRA · DEVOPS

AWS EC2 Linux Docker NginX Jenkins
Gitea Actions GitHub Actions Firebase Git

서버 운영 · SSL 적용 · DB 이관, CI/CD 구축 · 배포 자동화 · 런북
문서화

MOBILE · GAME

Flutter Capacitor Shorebird Play Console
App Store Connect Unity Mirror

Android · iOS 스토어 배포와 심사 대응, 코드푸시 운영, Unity
멀티플레이 · 네트워크

PAYMENT · ETC

토스페이먼츠 카카오페이 포트원 팜페이
WebRTC 공공 SI 산출물

PG 결제 연동 4종 경험 — 결제 · 승인 · 취소 · 환불 정책과 사전
점검 대응

01

CAREER 경력

공공 SI/SM에서 시작해 사내 업무 시스템 재구축, 그리고 자사 B2C 서비스와 고객사 플랫폼 운영까지.

(주)이코드

(주)유팜물

(주)TG

PG 결제 연동

데이터 처리 개선

배포 자동화

스토어 배포

(주)이코드

기술연구소 · 선임 | 2026.08 — 재직 중 · 3개월

재직 중

자사 B2C 서비스 · 고객사 플랫폼

● ROLE & RESPONSIBILITY

편지 우편 발송 플랫폼과 음성 AI 일기 앱 등 자사 B2C 서비스의 기능 개발 · 운영을 담당하고, 고객사 안전보건 플랫폼의 온보딩 · 검수 · 배포를 수행했습니다. 결제수단과 환불 정책을 PG 취소 기한에 맞춰 개선하고 우편 발송을 자동화했으며, 막혀 있던 자동 배포 파이프라인을 복구해 전구간 실배포로 검증하고 AWS 인벤토리 · 백업 · 배포 런북을 문서화했습니다.

- Next.js
- NestJS
- Flutter
- PostgreSQL
- Redis
- AWS EC2
- Gitea Actions
- PG 결제
- iOS / Android 배포

● KEY HIGHLIGHTS

- 01 PG 결제 · 환불 정책 개선
0원 PG 결제 제거 · 취소 기한 기반 환불 선차단
- 02 우편 발송 자동화
예약 해제분 인쇄 수거 · 우체국 추적 1일 2회
- 03 자동 배포 파이프라인 복구
EC2 + SSM 전환 · 전구간 실배포 검증 · 런북 문서화
- 04 고객사 3곳 온보딩
인증서 발급 · 계정 · 운영 점검 · 4환경 배포
- 05 Android · iOS 스토어 배포
targetSdk 36 · App Privacy · Apple 심사 2.1 대응
- 06 사내 메일 고도화 · 연차 산정
주소록 · 서명 · 발송 취소 · 근로기준법 60조 기준

결제 · 환불 정책 개선과 우편 발송 자동화

편지 인쇄 · 우편 발송 플랫폼 — 결제수단 분리와 환불 정책 확장

토스페이먼츠

카카오페이

NestJS

문제

적립금을 100% 사용한 주문도 PG 결제창을 거쳐 **0원 결제**가 발생했고, 결제 후 오래된 주문은 PG 자동 취소 기한(카드 364일, 계좌이체 · 간편결제 179일)을 넘겨 환불이 실패할 수 있었습니다. 예약 발송 건의 인쇄 수거와 우체국 추적 조회도 수동 작업이었습니다.

설계 · 구현

적립금 전액 결제를 PG를 거치지 않는 자체 결제수단으로 분리하고 기존 주문을 마이그레이션으로 백필했습니다. 환불 정책 계산에 PG 취소 기한을 넣어 기한 경과 건은 **DB 커밋 전 409로 선차단**하고 마이페이지에서 계좌 환불 경로를 안내했으며, 약관 제10조와 운영 문서를 갱신하고 테스트를 추가했습니다. 예약 해제분 자동 인쇄 수거와 우체국 XML 응답 파싱, 추적 조회 1일 2회 스케줄을 구현했습니다.

결과 · 역량

결제수단과 환불 정책을 안전하게 확장해 **0원 PG 결제와 환불 실패 민원**을 제거했고, 우편 발송의 수동 작업을 없앴습니다. 다른 서비스의 PG(포트윈) 사전 점검 지적도 공개 사업자 정보 페이지 추가로 당일 해소했습니다.

● 환불 요청 처리 흐름

환불 요청



결제수단 · 승인일 조회



PG 취소 기한 계산
카드 364일 · 간편결제 179일



기한 경과 → 409 선차단

DB 커밋 전에 막아 실패 트랜잭션을 만들지 않는다



마이페이지에서 계좌 환불 안내

자동 배포 파이프라인 복구와 인프라 문서화

막혀 있던 CI/CD를 되살리고 인수인계 가능한 상태로 정리

Gitea Actions

AWS EC2

SSM

문제

자동 배포 파이프라인이 막혀 **수동 배포에 의존**하고 있었고, 관리되지 않는 terraform과 미사용 AWS 리소스, 문서화되지 않은 백업 · 배포 키가 운영 리스크로 남아 있었습니다.

설계 · 구현

Gitea Actions 러너를 운영 EC2로 옮겨 액세스 키 방식으로 재구성하고, main 배포 워크플로를 **EC2 + SSM 방식**으로 교체해 앱별 변경 감지로 선택 배포하도록 했습니다. 막혀 있던 원인 5가지를 수정한 뒤 terraform 폐기 절차와 AWS 인벤토리 · 배포 런북 · DB 백업 · ECR 수명주기 · 로그 회전 기준을 문서화했습니다.

결과 · 역량

CI 파이프라인 green과 api 실패포로 **전구간을 검증**했고, 다른 PC에서도 러너를 띄울 수 있도록 인수인계 문서를 남겼습니다.

BEFORE → AFTER

BEFORE

수동 배포 · 관리되지 않는 인프라

파이프라인 중단, terraform 방치, 미사용 리소스, 백업 · 키 미문서화



AFTER

EC2 + SSM 자동 배포 · 런북 정비

앱별 변경 감지 선택 배포, terraform 폐기 기록, AWS 인벤토리 · DB 백업 · ECR 수명주기 · 로그 회전 기준 문서화

5가지

막혀 있던 원인 수정

전구간 실패포로 검증

6회

운영 배포 수행

104 — 109회차

고객사 안전보건 플랫폼 운영 · 온보딩

회사별 빌드 이미지 운영과 파일럿 고객사 온보딩

Next.js

RDS

Docker · EC2

문제

고객사 안전보건 플랫폼을 여러 회사가 **각각의 빌드 이미지**로 사용하는 구조에서, 파일럿 온보딩과 운영 배포, 검수, 고객 회신이 동시에 진행되어야 했습니다.

설계 · 구현

회사별 빌드 브랜치 체계를 정리하고 main 변경을 각 브랜치에 반복 머지했으며, 데모 · 운영 이미지의 시드와 데이터 캡처 게이트를 빌드 인자로 분리했습니다. 파일럿 고객사 3곳의 개설 (인증서 발급 · 부착 · 외부 검증, 계정 발급, 운영 점검 22/22)과 4개 환경 배포, 백업 후 파일럿 데이터 삭제 · 본계약 전환 검수, 회사별 · 일별 DB 사용량 추이 도구를 수행했습니다.

결과 · 역량

약 2개월간 검수 · 배포 · 고객 회신 기록 **40건**을 남기며 회사별 운영 이미지가 끊기지 않도록 유지했습니다.

● 운영 지표

3곳

파일럿 온보딩

인증서 · 계정 · 점검

4환경

동시 운영 배포

스테이징 · 운영

109회

브랜치 동기화

운영 3사 + 데모

● 회사별 빌드 브랜치 운영

main — 공통 기능 개발

↓ 반복 머지 109회

build/A사

build/B사

build/C사

build/demo

시드 · 데이터 캡처 게이트를 빌드 인자로 분리해 이미지별로 다르게 구성

Android · iOS 스토어 배포와 Apple 심사 대응

제출 자료 작성부터 심사 정보 요청 대응까지 한 사이클

Capacitor

Play Console

TestFlight

문제

고객사 안전보건 웹앱을 Capacitor WebView로 감싼 스토어 앱에 Android 릴리스 구성이 없었고, iOS는 App Store 제출 자료와 Apple 심사 정보 요청(계정 삭제 5.1.1, UGC 신고 · 차단 1.2, B2B 배포 3.2) 대응이 필요했습니다.

설계 · 구현

Android 릴리스 서명을 구성하고 **targetSdk 34 → 36**으로 올리고 WebView 뒤로가기 처리와 Play 아이콘 · 스크린샷을 준비했습니다. iOS는 App Store Connect 메타데이터와 App Privacy 설문, 규격 스크린샷(iPhone 1284×2778 · iPad 2048×2732)을 작성하고, 심사 정보 요청에는 실기기 화면 녹화 18단계 시나리오와 영문 회신 · Review Notes로 대응했습니다.

결과 · 역량

계정 삭제 요건에 맞춰 플랫폼에 **회원 탈퇴 신청 기능**과 마이그레이션을 추가해 심사 요건을 코드로 반영했고, Android · iOS 배포 절차와 재발 방지 기준을 문서로 남겼습니다.

● 심사 대응 항목

01 계정 삭제 (5.1.1 v)

더보기 > 회원 탈퇴 신청 · 탈퇴 API · 마이그레이션 017 · 모바일 셀 UI

02 UGC 신고 · 차단 (1.2)

메신저 차단 경로 점검 및 차단 해제 흐름 결함 확정 · E2E 검증

03 B2B 배포 (3.2)

Unlisted 배포 대안 검토 및 회신

04 제출 자료

앱 이름 · 부제 · 설명 · 키워드 · 지원/개인정보 URL · 심사 정보(데모 계정) · 수출 규정

05 App Privacy 설문

수집 항목별 근거 정리 · 미수집 항목 코드 확인

음성 AI 일기 앱 리뉴얼과 스토어 릴리스

Flutter 앱 — 음성 인식 안정화와 전면 UI 리뉴얼

Flutter

Riverpod

Shorebird

문제

음성 인식 텍스트가 유실되고 **세션 재시작이** 폭주했으며, iOS 2분 녹음 제한 후 이어 녹음하면 앞 내용이 중복 저장되고 데일리 알림이 오지 않는 등 핵심 사용 경로에 결함이 있었습니다.

설계 · 구현

빈 부분결과가 본문을 지우던 원인을 제거하고 **pauseFor를 120초 → 8초**로 줄이고 error_busy 연쇄 재시작에 백오프를 넣었습니다. 하단바를 5탭으로 리뉴얼하고 작성 화면 · 공통 위젯 · 감정 아이콘을 전면 정리했으며, PDF 내보내기과 글자 크기 · 스타일 설정, 업데이트 안내 다이얼로그를 추가했습니다. Shorebird 코드푸시를 도입해 Android는 스토어 심사 없이 패치를 배포했습니다.

결과 · 역량

1.0.9+50 → 1.0.10+54 빌드 5개를 Play Console · TestFlight로 릴리스하고 코드푸시 패치 3회를 배포했으며, 릴리스 이력과 재발 방지 절차를 문서로 관리했습니다.

● 주요 개선

01 음성 인식 유실 · 재시작 폭주

빈 부분결과 제거 · pauseFor 120→8초 · 백오프

02 iOS 2분 cap 중복 저장

이어 녹음 시 앞 내용 중복 · 저장 오류 수정

03 데일리 알림 미수신

예약 알림 리시버 추가

04 전면 UI 리뉴얼

5탭 하단바 · 작성 화면 개편 · 공통 위젯 · 디자인 토큰 통합

05 기능 추가

PDF 내보내기 · 글자 크기/스타일 · 업데이트 안내

5빌드

스토어 릴리스

Play · TestFlight

3회

코드푸시 패치

심사 없이 배포

사내 업무 솔루션 메일 고도화와 연차 산정 개선

웹 · API · 모바일 세 패키지를 오가며 기능을 끝까지

React

Cloudflare Workers · D1

Flutter

문제

사내 통합 업무 솔루션의 메일 기능이 **기본 송수신 수준**이라 주소록 · 서명 · 발송 취소 같은 업무 메일 기능이 없었고, 연차 산정이 법정 기준과 맞지 않았습니다.

설계 · 구현

개인 · 공통 주소록과 CSV 가져오기 · 내보내기, HTML 서명과 CID 인라인 로고 발송, 보내기 대기 · 발송 취소, 답장 인용 세로줄, 목록 접기 · 폭 조절, 목록 유지 새로고침 · 실행 취소를 구현하고 외부 HTML 메일의 **스크립트 실행(XSS)**을 차단했습니다. 연차 산정을 근로기준법 60조 기준 입사일 계산으로 교체하고 입사일 기본값을 가입일과 연동했습니다.

결과 · 역량

사내 메일을 업무용 클라이언트 수준으로 끌어올렸고, 다크모드와 알림 개인 설정을 웹 · 앱 양쪽에 도입해 Android 앱을 0.5.1 → 0.6.2까지 사내 배포했습니다.

구현 항목

01 주소록

개인 · 공통 관리 + CSV 가져오기 / 내보내기

02 HTML 서명

회사 추천 서명 · CID 인라인 로고 발송

03 발송 제어

보내기 대기시간 설정 · 어디서든 발송 취소

04 읽기 경험

답장 인용 세로줄 · 목록 접기 · 폭 끌기

05 보안

외부 HTML 메일 스크립트 실행 차단

06 연차 산정

근로기준법 60조 기준 · 입사일 기본값 = 가입일

(주)유팜물

의약품 도 · 소매 중계 | 개발팀 · 주임 | 2025.03 — 2026.07 · 17개월

ASP.NET

C#

MSSQL

ERP 연동

PG 결제

● ROLE & RESPONSIBILITY

웹 기반 업무 시스템의 기능 개선과 운영 고도화를 담당하며, 요구사항 분석부터 설계 · 개발 · 이슈 대응까지 수행했습니다. 팜페이 결제 PG API를 시스템에 맞춰 신규 연동하고, WOS 업무 화면을 디자인 · 연동 기능까지 전면 재구축했으며, MSSQL 프로시저 · 대용량 거래 데이터 처리 개선과 ERP DB 연동 안정화 · 외부 데이터 수집으로 운영 데이터 활용 범위를 확장했습니다.

99→70%

피크 CPU 사용률
실행계획 · 인덱스 정비

70%

2,000명 동시접속
개선 후 안정 구간

45%

1,200명 동시접속
개선 후 여유 수준

● KEY HIGHLIGHTS

- 01 팜페이 결제 PG 연동**
전 과정 구축 — 요청 · 승인 · 취소
- 02 MSSQL 프로시저 개선**
대용량 거래 조회 응답 개선 · 피크 CPU 완화
- 03 WOS 화면 전면 재구축**
프론트 신규 구축 + 백엔드 기능 확장
- 04 ERP DB 연동 안정화**
장애 대응 · 정합성 점검 체계 정비
- 05 외부 데이터 수집 · 분석**
시스템 신규 개발 · 운영 데이터 활용 확장

운영 시스템 개선 사례 *before* → *after*

화면 구조 · 조회 응답성 · 운영 안정성

● 01 · 디자인 · 연동 전면 재구축

WOS 화면 전면 재구축

BEFORE

한 화면 구조 · 일관성 부족

디자인 · 정보 구조가 화면마다 제각각이고 연동 기능도 분산

AFTER

프론트 전면 재구축 + 백엔드 확장

요구사항 기반으로 디자인 · 연동을 새로 설계하고 필요한 백엔드 기능 추가 · 확장

● 02 · 조회 응답성 개선

프로시저 · 대용량 거래 데이터

BEFORE

느린 조회 · 서버 응답 지연

복잡한 프로시저로 대용량 처리 시 지연 발생, 피크 CPU 99~100%

AFTER

실행 계획 기반 쿼리 · 인덱스 개선

병목 구간을 찾아 조회 응답 지연 완화, CPU 70% 수준 안정화

● 03 · 운영 안정성 확보

ERP DB 연동 안정화

BEFORE

장애 시 수동 점검 · 복구

ERP DB 장애 발생 시 즉시 대응 어려움

AFTER

연동 점검 + 대응 체계 정비

점검 · 복구 절차 표준화로 장애 대응 시간 단축

MSSQL 프로시저 · 대용량 거래 데이터 개선

실행 계획 분석으로 병목을 찾아 프로시저 · 인덱스를 정비

MSSQL

실행 계획

인덱스

프로시저

● 개선 접근

01 실행 계획 분석으로 병목 구간 식별

비용이 큰 조희 구간을 우선순위로 정리

02 쿼리 구조 · 인덱스 정비

조희 조건과 인덱스 정합을 점검해 불필요한 비용 제거

03 프로시저 단계적 개선 · 검증 후 반영

기존 조희 결과와 비교 검증하며 운영 영향 없이 적용

거래 데이터가 누적되며 주요 조희 화면의 응답이 느려지고, 피크 시 DB CPU가 99~100%까지 치솟아 **응답 지연 · 장애 위험**이 있었습니다.

● 성능 통계 · DB CPU 사용률

동시 접속 부하 테스트 기준

개선 전 · 피크 부하

조희 폭주 시 CPU 한계 도달 → 응답 지연 · 장애 위험

99~100%

개선 후 · 2,000명 동시 접속

안정 구간 유지

70%

개선 후 · 1,200명 동시 접속

여유 있는 부하 수준 유지

45%

WOS 화면 전면 재구축 · 팜페이 결제 PG 연동

디자인 · 연동 기능 재설계 + 백엔드 기능 확장

ASP.NET MVC

UI/UX

PG API

● WOS 화면 전면 재구축

노후한 WOS 화면을 요구사항 기반으로 디자인 · 정보 구조부터 연동 기능까지 새로 설계하고, 필요한 백엔드 기능도 추가 · 확장하며 신규 구축 수준으로 재구축했습니다.

01 화면마다 제각각인 디자인 · 정보 구조

→ 공통 컴포넌트 · 레이아웃 기준 정의로 화면 일관성 확보

02 분산된 연동 기능과 노후 구조

→ 연동 흐름을 정리하고 화면-서버 연동을 새로 구현

03 운영 중 시스템 — 점진적 전환 필요

→ 백엔드는 유지 · 확장하면서 프론트를 안전하게 교체

● 팜페이 결제 PG API 연동

문제

의약품 도 · 소매 거래 대금 결제를 위해 팜페이가 제공하는 결제(PG) API를 업무 시스템에 신규로 연동해야 했습니다.

설계 · 구현

팜페이 PG API 문서(스펙)를 분석해 결제 요청 · 승인 · 취소 등 필요한 API를 시스템 구조와 데이터 흐름에 맞게 연동 · 적용하고, 결제 결과를 주문 · 거래 데이터와 연결해 처리했습니다.

결과 · 역량

외부 PG 결제 연동을 시스템에 적용해 거래 결제를 처리할 수 있게 했고, 외부 API 문서를 해석해 기존 시스템에 맞춰 연동하는 경험을 확보했습니다.

ERP DB 연동 안정화 · 외부 데이터 수집

장애 대응 체계 정비와 운영 데이터 활용 확장

ERP 연동

정합성 점검

데이터 수집

● 과제

01 ERP DB 장애 시 수동 점검 · 복구

장애 발생 시 원인 파악과 복구에 즉시 대응이 어려움

02 연동 데이터 정합성 관리 필요

업무 시스템과 ERP 간 데이터 불일치 시 업무에 직접 영향

03 운영 데이터 활용 범위 확장 필요

운영 판단에 필요한 외부 데이터의 수집 · 분석 요구

● 구현 · 성과

01 연동 점검 · 복구 절차 표준화

장애 유형별 대응 절차를 정리해 대응 속도 개선

02 정합성 점검 체계 정비

불일치를 조기에 발견하는 점검 흐름 구성

03 외부 데이터 수집 · 분석 시스템 신규 개발

운영 데이터 활용 범위 확장

운영 안정성 향상

ERP 연동 오류 대응과 데이터 관리 부담 완화

(주)TG

플랫폼팀 · 주임 | 2022.03 — 2024.02 · 약 2년

- React.js
- ReactQuery
- OpenAPI
- 공공 SI

공공 SI/SM과 서비스형 플랫폼 개발에 참여하며 관리자 시스템, OpenAPI, 공공기관 유지관리, React 기반 화면 구현과 운영 개선을 수행했습니다.

PROJECTS · 회사에서 참여한 프로젝트

2022.03 — 2022.12	인천스마트관광도시 앱 서버 · 관리자 고도화 서버 · 관리자 · API 개발 · 고도화 + 관광지 · 주차장 OpenAPI (앱은 협력사 협업)	Backend · API 9개월
2023.01 — 2024.02	정보화통합관리체계 운영개선 및 유지관리 공공기관 SI/SM · 유지보수 · 보안 조치 · 서버/DB 이관 · 데이터 현행화	JSP 14개월
2022.12 — 2023.03 2023.10 — 2024.02	다지점 동물 임상시험 플랫폼 구축 · 고도화 React 기반 신규 구축 · 적응형 UI · ReactQuery 기반 운영 고도화 · 매뉴얼 작성	Frontend · UI/UX 총 9개월
2023.06 — 2023.08	동물 원격진료 플랫폼 MVP 개발 React.js · ReactQuery 프론트엔드 + WebRTC 영상통화 서버 연동 · 실시간 채팅 UI	Frontend 3개월

인천스마트관광도시 앱 서버 · 관리자 고도화

2022.03 — 2022.12 · 9개월 | 서버 · 관리자 · API 담당 (앱은 협력사와 협업)

서버 · API

관리자 시스템

OpenAPI

● 과제

01 앱 서버 · 관리자 기능 고도화 필요

서버 · 관리자 · API를 당사가 개선 · 확장 (앱 클라이언트는 협력사)

02 콘텐츠 운영 흐름 개선 필요

관리자에서 관광 콘텐츠를 더 효율적으로 관리할 수 있어야 함

03 운영 데이터를 외부와 연계할 OpenAPI 필요

관광 데이터를 외부 서비스에서 활용할 수 있도록 제공

● 구현 · 성과

01 서버 · 관리자 신규 기능 개발 · 고도화

앱 서버 로직과 관리자 페이지 기능 개선 및 신규 기능 추가

02 운영 이슈 디버깅 · 협력사 대응

백오피스 운영 효율 개선, 연동 이슈 협업

03 관광지 · 주차장 OpenAPI 설계 · 개발

관광 데이터를 외부 서비스에 제공하는 OpenAPI 구축 (후속)

04 공공 SI 산출물 작성

기능 명세서 · 테이블 정의서 · 프로그램 명세서

동물 임상시험 플랫폼 · 원격진료 MVP

신규 구축 이후 사용자 피드백과 운영 흐름을 반영해 고도화

React.js

ReactQuery

WebRTC

● 다지점 동물 임상시험 플랫폼 · 총 9개월

01 분산된 임상시험 운영 업무 통합

계획 의뢰 · 진행 관리 · 참가 개체 · 서류 · 리워드를 웹 기반으로 통합

02 PC / Mobile / Tablet 적응형 UI

환경에 관계없이 일관된 사용 흐름 확보

03 ReactQuery 도입 · 고도화

서버 상태 관리와 데이터 갱신 흐름 개선, 중복 요청 감소

04 관리자 / 사용자 매뉴얼 작성

온보딩과 문의 대응 효율 개선

● 동물 원격진료 플랫폼 MVP · 3개월

문제

동물병원과 보호자 간 비대면 상담 흐름을 검증하기 위한 서비스형 MVP가 필요했습니다.

설계 · 구현

React.js와 ReactQuery 기반 프론트엔드를 구현하고, 백엔드에서 구축된 **WebRTC 영상통화 서버**와 연동해 화상 상담 및 실시간 채팅 UI를 구성했습니다. 기획 · 디자인 시안을 화면으로 퍼블리싱했습니다.

결과 · 역량

원격 상담의 핵심 사용자 흐름을 프로토타입으로 완성해 신규 서비스 검증 기반을 마련했습니다.

정보화통합관리체계 운영개선 및 유지관리

2023.01 — 2024.02 · 14개월 | 공공 SI/SM · 유지관리 / 운영지원

공공 SI/SM

유지보수

보안 조치

● 과제

01 다수 공공기관 납품 시스템의 안정적 운영

고객 요구사항 · 장애 · 오류에 대한 지속 대응 필요

02 보안 취약점 조치와 정기 점검

서버 · DB 이관 및 운영 데이터 현행화 필요

다수 공공기관에 납품된 시스템을 안정적으로 운영하면서 고객 요구사항, 보안 취약점, 서버 · DB 이관 등 운영 리스크에 대응해야 했습니다.

● 구현 · 성과

01 요구사항 분석 · 기능 개선 · 오류 수정

유지관리 작업 수행

02 보안 패치 및 취약점 조치

점검 결과 기반 보안 패치

03 정기 점검 · 서버 · DB 이관 운영 지원

운영 환경 전환 과정의 리스크 완화

04 운영 데이터 현행화 · 시스템 데이터 관리

데이터 정확성과 관리 체계 개선

공공 시스템 서비스 지속성과 보안 안정성 유지

02

WORKS 사이드 프로젝트

기획부터 배포까지 직접. 필요해서 만들고, 만들면서 구조를 다시 생각한 결과물입니다.

PDF Converter

Portfolio Site v2

NEKO PARK

좀비, 평평! 화르륵

TheWildFour

프로젝트 요약 *selected*

개인 2건 · 팀 3건

● PROJECT · 01

2주 · 2025.11

PDF Converter

서버 전송 없이 클라이언트 사이드에서 다양한 이미지를 PDF로 변환

Next.js

Tailwind CSS

Zustand

pdf-lib

Firebase Hosting

WEB · SOLO · pdf-converter-18eb5.web.app

● PROJECT · 02

4개월 · 2025.12 — 2026.03

Portfolio Site v2

Firestore 기반 동적 포트폴리오 운영 구조

Next.js

TypeScript

Tailwind CSS

Firebase

Motion

WEB · SOLO · sujin-portfolio.web.app

● PROJECT · 03

8일 · 2024.08

NEKO PARK

Mirror 세션과 분리된 TCP 방 목록 서버로 협력 퍼즐 게임의 네트워크 진입 흐름 구현

Unity

C#

Mirror

TCP

SHA-256

GAME · TEAM

● PROJECT · 04 · 05

2024.07 — 12

좀비, 평평! 화르륵 · TheWildFour

원소 상호작용 퍼즐 + AWS 백엔드 인프라 / 4인 협동 생존 게임의 아이템 · 인벤토리 구조

Unity

C#

Spring Boot

MariaDB

Docker

AWS

GAME · TEAM

PDF Converter

2025.11.04 — 2025.11.21 (2주) | 서버 업로드 없이 브라우저 내부에서 이미지 정규화와 PDF 생성

Next.js

Zustand

pdf-lib

pdf-converter-18eb5.web.app

● 과제

01 개인정보 보호

서버 업로드 방식 변환기는 민감한 문서나 개인 사진 유출 우려가 있음

02 사용 흐름 단순화

광고 배너, 오클릭 유도 UI, 불필요한 로그인 절차가 변환 흐름을 방해

03 설치 없는 접근성

별도 앱 설치 없이 브라우저에서 바로 사용할 수 있는 변환 도구 필요

● 구현 · KEY FEATURES

01 클라이언트 사이드 파이프라인

이미지 로드부터 PDF 생성까지 브라우저에서 처리 — 파일이 외부 서버로 전송되지 않음

02 Drag & Drop · Folder Upload

리스트 내 재정렬 · webkitRelativePath 기준 자동 정렬

03 PDF Output Customization

A4 / A3 / A5 / Letter / Legal · 세로·가로 전환 · 화질 4단계 압축

04 단순한 UI

로그인 절차와 광고 요소 제거, 핵심 변환 흐름에 집중

UUID 기반 파일 식별

업로드와 Drag & Drop 재정렬에서 순서가 바뀌어도 같은 파일을 안정적으로 추적

● 초기 접근 — useRef 카운터

! ID 충돌 가능성

파일 / 폴더 업로드 탭이 독립적으로 ID를 생성하면 상위 병합 시 key 충돌

! 컴포넌트 결합도 증가

부모에서 ID를 생성하도록 강제하면 하위가 부모 로직에 의존

● 최종 선택 — crypto.randomUUID()

01 전역 고유 ID 보장

파일이 어느 컴포넌트에서 생성되든 충돌 없음

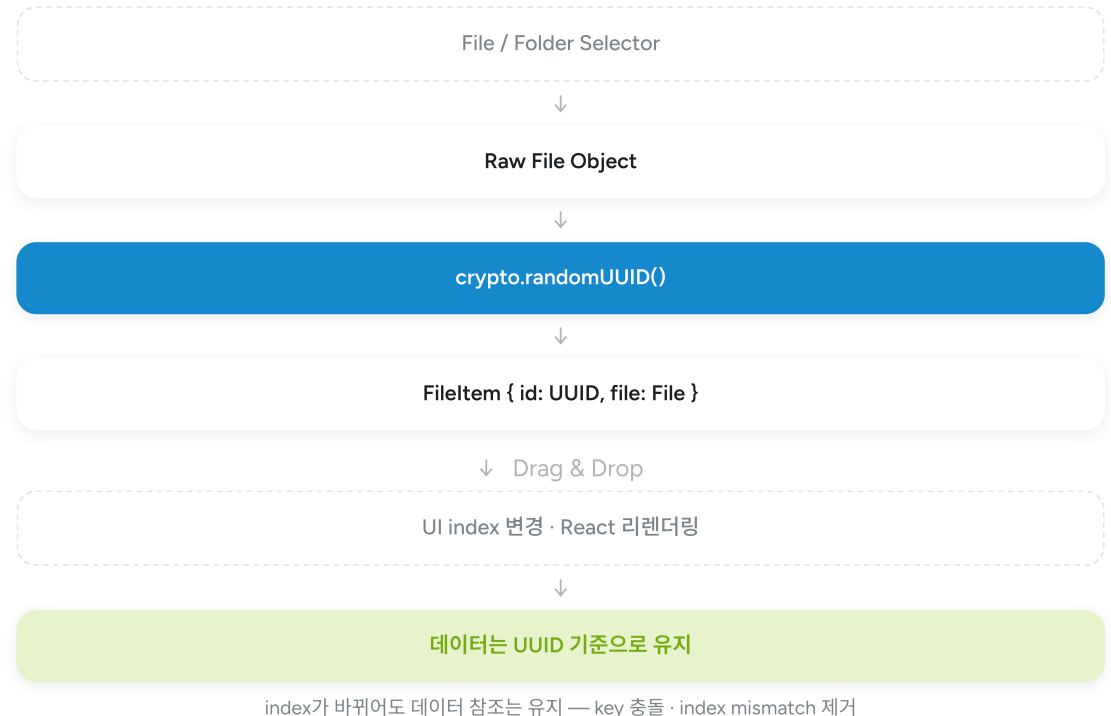
02 index 무관 추적

D&D 시 렌더링 index 변화와 무관하게 동일 파일 추적

03 컴포넌트 독립성 유지

별도 충돌 방지 로직 불필요, 이후 기능 확장에도 유리

● 데이터 흐름



탭 전환 시 상태 유지 · 폴더 업로드 정렬

상위 상태 관리 + Zustand 전역 상태 / 브라우저 보안 제약 대응

● 탭 전환 시 파일 상태 유지

! 문제 — 탭 내부 상태 관리

탭 이동 시 컴포넌트가 Unmount → Mount 반복되며 선택한 파일 목록과 정렬 상태가 초기화

✓ 해결 — 상위 상태 관리

파일 관련 상태를 상위 영역과 Zustand store에서 일관 관리, 개별 탭은 UI 역할에 집중

BEFORE — Tab A [state] / Tab B [state]

↳ 탭 전환 시 데이터 소실



AFTER — Home / Store [file state]

Tab A (UI only) / Tab B (UI only)

✓ 탭 전환에도 파일 목록 유지

● 폴더 업로드 — 경로 기반 정렬

! 브라우저 보안 정책

절대 경로와 실제 파일 시스템 정보 저장 불가. 순서가 무너지면 PDF 페이지 흐름도 달라짐

✓ 파일 목록 정규화

선택 시점에 공통 데이터 모델로 정규화하고, webkitRelativePath 기반 경로 정렬로 폴더 구조와 같은 순서 유지

✓ 폴더 단위 요약 렌더링

화면 부담 축소

폴더 업로드



브라우저 보안 정책 — 절대 경로 저장 불가



파일 목록 정규화



webkitRelativePath 기반 정렬

✓ 폴더 구조 · 순서 유지

EXIF 방향 문제와 Canvas 정규화 파이프라인

화면 표시가 아니라 픽셀 데이터 자체를 정상 방향으로 재생성

● 초기 접근 — ArrayBuffer 직접 삽입

! 현상

ArrayBuffer를 그대로 PDF에 삽입하면 EXIF 방향 보정이 반영되지 않음

! 구조적 원인

pdf-lib 변환 과정에서 EXIF 메타데이터 기반 방향 정보가 유지되지 않음

! 리스크

동일 파일도 기기 · 브라우저에 따라 다른 방향으로 출력



● 해결 — createImageBitmap + Canvas

01 createImageBitmap(file)

EXIF가 반영된 비트맵을 생성해 방향 차이 흡수

02 Canvas 재드로우

재렌더링으로 픽셀 방향 고정

03 JPEG 재인코딩

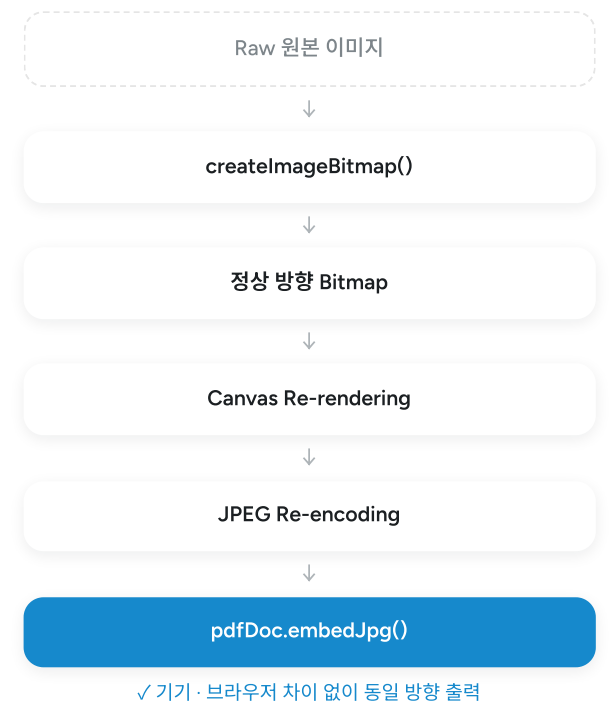
정규화 이미지로 재인코딩 후 PDF 삽입

04 ArrayBuffer Fallback

정규화 실패 환경에서도 변환이 중단되지 않도록 대비

다운스케일링과 Canvas 재사용으로 처리 비용도 완화했습니다.

● 정규화 파이프라인



결과 및 배운 점 *takeaway*

01

클라이언트 파일 처리 흐름 설계

이미지 업로드 → 정규화 → PDF 출력까지 서버 전송 없는
처리 파이프라인을 직접 설계했습니다.

02

EXIF 방향 이슈 해결

기기별로 달라지는 Orientation 문제를 Canvas 기반
정규화로 처리했습니다.

03

UUID 기반 데이터 무결성

Drag & Drop 재정렬 시 index가 아닌 파일 고유 ID로
안정적인 렌더링 구조를 구성했습니다.

Portfolio Site v2

2025.12 — 2026.03 (4개월) | 기획 / 디자인 / 개발 · Firestore 기반 동적 포트폴리오 운영 구조

Next.js

TypeScript

Firebase

sujin-portfolio.web.app

● 과제

01 기존 GitHub Pages 기반 정적 포트폴리오 운영

서버 기능을 활용하기 어려워 동적 콘텐츠 확장에 제약

02 콘텐츠 갱신마다 코드 수정 · 재배포 필요

프로젝트 · 경력 · 기술스택 정보가 늘어날수록 운영 부담 누적

03 데이터 기반 화면 확장 구조 필요

정적 호스팅 제약으로 콘텐츠 관리와 화면 로직이 강하게 결합

● 구현

01 Firebase Hosting + Firestore 동적 콘텐츠 구조

프로젝트 · 경력 · 기술스택 데이터를 Firestore 컬렉션으로 분리

02 전용 관리자 페이지 구축

코드 수정 및 재배포 없이 콘텐츠 등록 · 수정 · 삭제 가능

03 화면 · API 레이어 · 데이터 변환 로직 분리

유지보수성과 확장성을 고려한 운영 구조 구현

정적 배포 → 데이터 운영

Firebase Hosting + Firestore 전환과 데이터 기반 콘텐츠 관리

● Before · GitHub Pages 정적 호스팅

- ! 정적 빌드 결과물 배포 — 콘텐츠와 화면 코드가 결합되어 수정 비용 증가
- ! 동적 데이터 조회와 관리자 입력 흐름 확장 제약
- ! 콘텐츠 갱신마다 코드 수정 · 재배포 필요

● After · Firebase Hosting + Firestore

- ✓ Firestore 조회 구조 활성화 — career · project · skill · docs 컬렉션 단위 관리
- ✓ 프로젝트 상세는 역할 · 트러블슈팅 · 성과 블록을 재사용 가능한 스키마로 구성
- ✓ 전용 관리자 페이지 — Firestore 콘솔 접근 없이 콘텐츠 관리, 입력 즉시 반영
- ✓ 운영 부담을 코드 배포에서 데이터 관리로 전환

● 구조



티스토리 RSS 연동

CORS · 정적 빌드 · 외부 프록시 문제를 거쳐 빌드 시점 서버 fetch로 정리

● 시도와 선택

1 클라이언트 직접 fetch

브라우저 보안 정책(CORS)으로 Tistory RSS 직접 요청 차단

2 API Route 사용

정적 export(output: 'export') 구조와 API Route 사용이 충돌

3 무료 CORS 프록시

응답 불안정 — 실제로 프록시 4곳이 모두 중단되어(403 · 522 · 연결 실패)
글이 하나도 노출되지 않는 장애 발생

4 최종 — 빌드 시점 서버 fetch

페이지를 서버 컴포넌트로 분리해 빌드 때 RSS를 직접 가져와 HTML에 포함.
정적 export를 유지하면서 CORS · 프록시 · 로딩 상태를 모두 제거

● 최종 구조

× Client → Tistory RSS (CORS)

× Client → API Route (export 충돌)

× Client → CORS Proxy (불안정)



Build → Server fetch → RSS



정적 HTML에 글 목록 포함

✓ 프록시 · 로딩 상태 불필요 · 배포마다 갱신

구현 디테일 *details*

● 데이터 · 렌더링

01 Dynamic Route + SSG

경력 · 프로젝트 상세 경로를 사전 생성

02 Markdown-like 텍스트 강조

Firestore 텍스트의 ****강조**** 패턴으로 핵심 키워드 강조

03 라벨 파싱

문제 / 설계 · 구현 / 결과 · 역량 접두어를 구조화된 블록으로 변환

04 Firebase 중복 초기화 방지

getApps 체크로 빌드 · HMR 환경에서 인스턴스 중복 생성 방지

05 Image Visibility Guard

이미지 URL이 없거나 오류면 placeholder 없이 실제 이미지만 표시

● 디자인 시스템 · 인터랙션

01 다색 로테이션 토큰

부모에 색 클래스를 붙이면 CSS 변수가 바뀌어 점 · 알약 · 숫자 · 링크가 함께 따라옴

02 지표 자동 계산

경력 개월 수 · 회사 수 · 업무 건수를 데이터에서 계산. 기간이 겹쳐도 구간을 병합해 부풀지 않음

03 스크롤 섹션 감지

현재 섹션을 감지해 메뉴 활성 상태와 연결

04 모바일 레코드 전환

표를 라벨 + 값 카드로 접어 가로 스크롤 제거

05 Scroll to Top

조건부 버튼 노출 + 부드러운 상단 이동

팀 프로젝트 *unity*

네트워크 · 데이터 구조 · 인프라를 직접 설계한 게임 프로젝트

NEKO PARK

2024.08 · 8일

Mirror 기반 멀티플레이 세션에 TCP 방 목록 서버, 로그인, 로컬 세이브를 결합해 협력 퍼즐 게임의 네트워크 진입 흐름 구현

- ✓ Mirror 세션과 분리된 **TCP 방 목록 서버**로 방 생성 · 조회 · 입장 안정화
- ✓ TCP 승인 호스트를 Mirror RoomManager에 연결하는 2계층 구조
- ✓ SHA-256 해시 + Singleton 캐싱으로 계정 보안과 썬 전환 간 일관성
- ✓ JsonUtility 로컬 세이브로 오프라인 진행도 유지

좀비, 평평! 화르륵

2024.11 — 12

병합 퍼즐에 원소 속성 상호작용을 결합하고 Unity 클라이언트 · 리소스 운영 · AWS 백엔드를 함께 설계

- ✓ Physics2D 충돌 + 속성 테이블로 확장 가능한 원소 상호작용 설계
- ✓ Addressable + 외부 리소스 서버로 빌드 크기 · 런타임 메모리 최적화
- ✓ AWS EC2 · Docker Compose · Spring Boot · MariaDB REST API 구축
- ✓ 카탈로그 분리로 앱 재배포 없는 에셋 단위 업데이트

TheWildFour

2024.07 · 3주

4인 협동 3D 생존 게임의 아이템 데이터 구조, 범용 인벤토리, 맵 오브젝트 저장 · 복원 설계

- ✓ ScriptableObject 정적 데이터 분리로 인스턴스 중복 제거 · 메모리 효율 개선
- ✓ Observer Pattern 범용 인벤토리 — 플레이어 · 보관함 · 작업장에 재사용
- ✓ JSON 스키마 기반 맵 오브젝트 저장 · 복원
- ✓ Coroutine 비동기 처리로 시간 의존 로직을 프레임 저하 없이

Thank you

긴 시간 읽어주셔서 감사합니다

사용자 경험을 꼼꼼히 살피며
꾸준히 성장하는 개발자가 되겠습니다.

su_042@daum.net

github.com/Nuuuing

nuu-stradamus.tistory.com

sujin-portfolio.web.app